第 5 章　树和二叉树

5.1 引言

树形结构是一种典型的非线性数据结构，用它来描述客观世界中类似人类家庭族谱、单位各个分属部门等具有分支以及上下层次关系的结构非常方便。在计算机领域中，包括 UNIX 在内的许多常用操作系统的目录也使用树形结构。

树形结构的特点是，在数据元素的非空有限集合中：

（1）存在唯一一个被称为“根结点”的数据元素；

（2）存在若干个被称为“叶结点”（或终止结点）的数据元素；

（3）除根结点外，集合中每个元素均只有一个前驱；

（4）除叶结点外，集合中每个元素可以有若干个后继。

与数据之间存在一对一关系的线性结构相比，树形结构是一对多的非线性关系，即一个前驱可以对应多个后继。树形结构中最为常用的是二叉树，因此本章将重点讨论二叉树的逻辑表示、存储结构、常用操作，以及与树和森林之间的转换关系。

5.2 树的定义和基本术语

5.2.1 树的定义

定义树的一种自然的方式是递归。一棵**树**（tree）是n（$n \geq 0$）个结点的有限集T。若$n = 0$，则这个集合是空集；若$n > 0$，则树由称作**根**（root）的结点以及m（$m \geq 0$）个互不相交的有限子集$\{T_1, T_2, \ldots, T_m\}$组成。每一个子集本身又是一棵符合本定义的树，叫作根结点的**子树**。每一棵子树的根叫做根r的**孩子**（child），而r是每一棵子树的根的**双亲**（parent）。图 5.1 展示了用递归定义的典型的树。

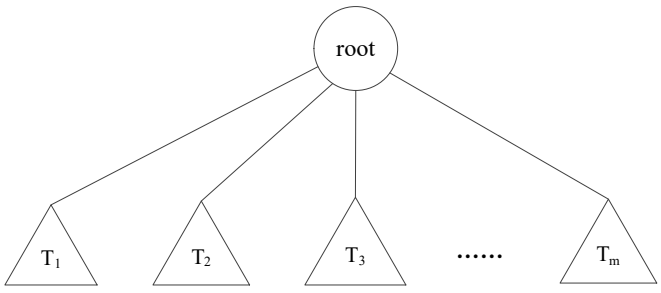


图 5.1　典型的树

由此可见，树的定义是递归的，因为在树的定义中又用到树的定义本身，即一棵树由若干子树构成，而子树又由更小的若干子树构成……因此树特别适合用来表示具有层次关系的数据结构。

从递归定义中可以发现，一棵树是n个结点和$n-1$个分支的集合，其中的一个结点是根。存在$n-1$个分支的结论是由下面的事实得出的——每个分支都将某个结点连接到它的双亲，而除去根结点外每个结点都只有一个双亲，如图 5.2 所示。

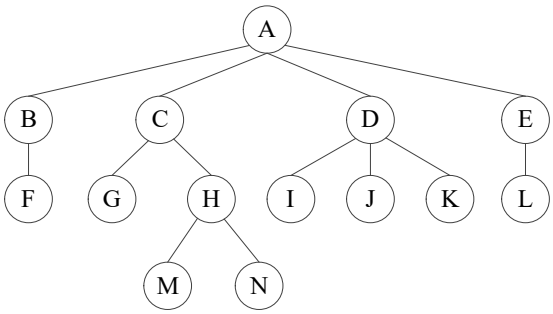


图 5.2 一棵具体的树

5.2.2 树的逻辑表示

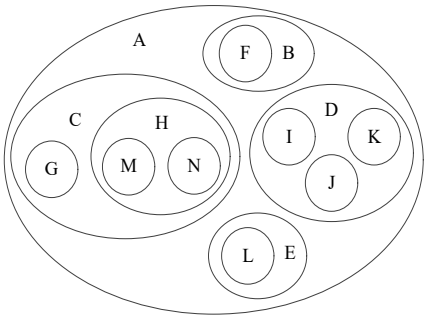
树的逻辑表示方法有多种，常见的逻辑表示方法如下：

（1）树形表示法：用圆圈表示结点，圆圈内的符号表示该结点的数据信息，结点之间的关系通过连线表示，如图 5.2 所示。

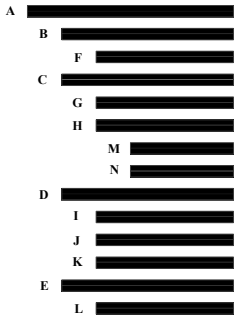
（2）文氏图表示法：每棵树对应一个圆圈，圆圈内包含表示根和子树的圆圈，各个子树对应的圆圈不能相交，如图 5.3a 表示的图 5.2 中的树。

（3）凹入表示法：每棵树的根对应一个条形，子树的根对应较短条形；同一个根下的各个子树的根对应的条形长度一致，如图 5.3b 表示的图 5.2 中的树。

（4）括号表示法：每棵树对应一个由根作为名字的表，表是由一个括号里的各个子树对应的子表构成，子表之间用逗号分开，如图 5.3c 表示的图 5.2 中的树。



a) 文氏图表示法



b) 凹入表示法

A(B(F),C(G,H(M,N)),D(I,J,K),E(L))

c) 括号表示法

图 5.3 树的各种表示方法

5.2.3 树的基本术语

在图 5.2 的树中，结点 A 是**根**。结点 D 有一个双亲 A 并且有三个孩子 I、J 和 K。每一个结点可以有任意多个孩子，也可以没有孩子。**结点的度**（degree）是指结点具有的子树（孩子）的个数。例如，结点 D 的度为 3，结点 E 的度为 1。度为 0 的结点称为**叶**（leaf）**结点**，也叫做**终止结点**；反之，度不为 0 的结点叫做**分支结点**。**树的度**定义为树中结点的度的最大值，例如，图 5.2 中树的度为 4。具有相同双亲的结点称为**兄弟**（brother）**结点**，因此 I、J 和 K 是兄弟。用类似的方法可以定义**祖先**（ancestor）和**子孙**（descendant）关系。

从结点n_1到n_k的**路径**（path）定义为结点n_1，n_2，…，n_k的一个序列，使得对于$1 \le i < k$，结点n_i是n_{i+1}的双亲。注意，在一棵树中，从根到每个结点都存在一条路径。**路径长度**（length）就是指路径中包含的边的数量。那么结点的**祖先**就是从根到该结点路径上经过的所有结点；而结

点的**子孙**就是以该结点为根的子树中的所有结点。例如，图 5.2 中 M 的祖先是 A、C 和 H；C 的子孙包括 G、H、M 和 N。

对任意结点n_i，其**层次**（level）是从根开始起定义的，即根为第一层，根的孩子是第二层，以此类推。树中结点所在的最大层次称为树的**深度**（depth）**或高度**，图 5.2 中的树的高度为 4。

森林（forest）是m（$m \geq 0$）棵互不相交的树的集合。对树中每个结点而言，其子树的集合即可视为森林。例如，在图 5.2 中，根结点 A 下有 4 棵子树构成的森林。

5.2.4 树的抽象数据类型

上述对树的定义和对树的一组基本操作构成了树的抽象数据类型定义。

```
ADT Tree
{
```

数据对象 D：D是具有相同特性的数据元素的集合。

数据关系 R：若D为空集，则称为空树；

若D仅含一个数据元素，则R为空集，否则$R=\{H\}$，H是如下的二元关系：

（1）若D中存在唯一的称为根的数据元素$root$，在它在关系H下无前驱；

（2）若$D-\{root\}\neq\Phi$，则存在$D-\{root\}$的一个划分$D_1, D_2, \cdots, D_m$（$m>0$），对任意$j\neq k$（$1\leq j,k\leq m$）有$D_j\cap D_k=\Phi$，且对任意的i（$1\leq i\leq m$），唯一存在数据元素$x_i\in D_i$，有$<root, x_i>\in H$；

（3）对应于$D-\{root\}$的划分，$H-\{<root,x_1>,\ldots,<root,x_m>\}$有唯一的一个划分$H_1, H_2, \cdots, H_m$（$m>0$），对任意$j\neq k$（$1\leq j,k\leq m$）有$H_j\cap H_k=\Phi$，且对任意$i$（$1\leq i\leq m$），$H_i$是$D_i$上的二元关系，$(D_i,\{H_i\})$是一棵符合本定义的树，称为根$root$的子树。

基本操作 P：

// 1. 初始化、销毁和清空操作

InitTree(&T)
　　操作结果：构造一个空树 T

DestroyTree(&T)
　　初始条件：树 T 存在
　　操作结果：销毁树 T

CreateTree(&T，definition)
　　初始条件：definition 给出树 T 的定义
　　操作结果：按 definition 构造树 T

ClearTree(&T)
　　初始条件：树 T 存在
　　操作结果：将树 T 重置为空树

// 2. 访问型操作

TreeEmpty(T)
　　初始条件：树 T 存在
　　操作结果：若树 T 为空树，则返回 TRUE，否则返回 FALSE

TreeDepth(T)
　　初始条件：树 T 存在
　　操作结果：返回 T 的深度

Root(T)
　　初始条件：树 T 存在
　　操作结果：返回 T 的根

Value(T, cur_p, &e)
　　初始条件：树 T 存在，cur_p 是 T 中某个结点的指针
　　操作结果：返回 cur_p 对应结点的元素值

Parent(T, cur_p, &parent)
　　初始条件：树 T 存在，cur_p 是 T 中某个结点的指针
　　操作结果：若 cur_p 非 T 的根结点指针，则返回它的双亲结点指针，否则返回值为空

LeftChild(T，cur_p, &leftChild)

初始条件：树 T 存在，cur_p 是 T 中某个结点的指针

操作结果：若 cur_p 是 T 的非叶结点指针，则返回它的最左孩子指针，否则返回值为空

RightSibling(T，cur_p, &rightSibling)

初始条件：树 T 存在，cur_p 是 T 中某个结点的指针

操作结果：若 cur_p 指向的结点有右兄弟，则返回它的第一个右兄弟指针，否则返回值为空

TraverseTree(T)

初始条件：树 T 存在

操作结果：按某种次序对 T 的每个结点访问（打印）一次且至多一次

// **3. 加工型操作**

Assign(&T，cur_p，value)

初始条件：树 T 存在，cur_p 是 T 中某个结点指针

操作结果：将结点 cur_p 的元素赋值为 value

InsertChild(&T， &p，i，c)

初始条件：树 T 存在，p 指向 T 中某个结点，$1 \le i \le p$ 所指结点的度加 1，非空树 c 与 T 不相交

操作结果：插入 c 为 T 中 p 指向结点的第 i 棵子树

DeleteChild(&T， &p，i)

初始条件：树 T 存在，p 指向 T 中某个结点，$1 \le i \le p$ 所指结点的度

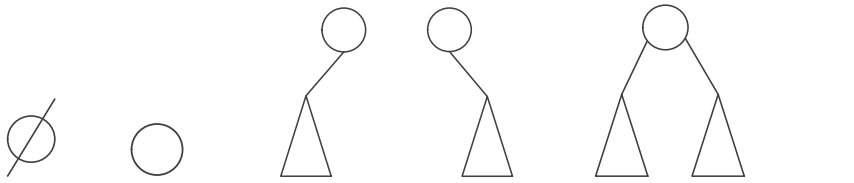
操作结果：删除 T 中 p 指向结点的第 i 棵子树

} // ADT Tree

5.3 二叉树

5.3.1 二叉树的定义

二叉树（binary tree）是一种每个结点最多只能有两个孩子的树。二叉树有 5 种基本形态，如图 5.4 所示，任何复杂的二叉树都是这 5 种基本形态的复合。其中图 5.4a 是空树，图 5.4b 是仅有根结点的二叉树，图 5.4c 是仅有左子树（右子树为空）的二叉树，图 5.4d 是仅有右子树（左子树为空）的二叉树，图 5.4e 是左右子树都不空的二叉树。5.2 节中引入的有关树的基本术语也都适用于二叉树。

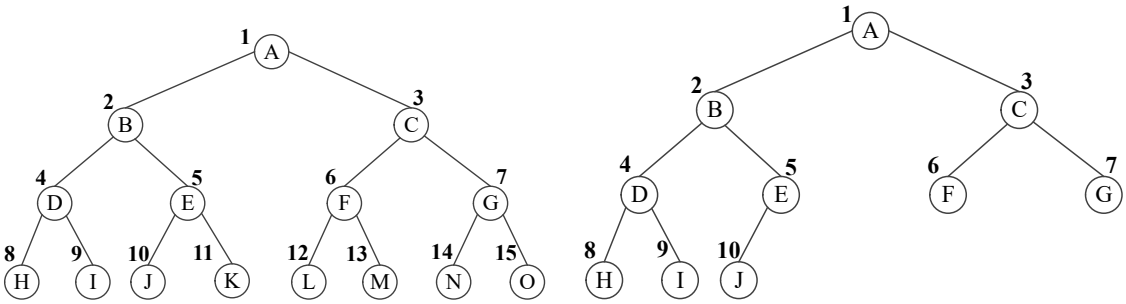


a)空树　b) 仅有根结点　c)仅有左子树　d) 仅有右子树　e) 既有左子树也有右子树

图 5.4　二叉树的 5 种基本形态

在一棵二叉树中，若所有分支结点的度均为 2，并且所有的叶子结点都集中出现在二叉树的最底层，这样的二叉树称为**满二叉树**，如图 5.5a 所示的就是一棵满二叉树。

若二叉树中度小于 2 的结点只能出现在树的最下面两层，而且最底层的叶结点都依次排列在该层最左边的位置上，则这样的二叉树称之为**完全二叉树**，如图 5.5b 所示的就是一棵完全二叉树，完全二叉树中结点的编号与满二叉树结点编号一一对应。不难看出，满二叉树是完全二叉树的一种特例。



a)满二叉树　　b) 完全二叉树

图 5.5 满二叉树与完全二叉树示例图

（1） 对应于$D-\{root\}$的划分，$H-\{<root,x_1>,\ldots,<root,x_m>\}$有唯一的一个划分$H_1$，$H_2$，$\cdots$，$H_m$（$m>0$），对任意$j\neq k$（$1\leq j,k\leq m$）有$H_j\cap H_k=\Phi$，且对任意$i$（$1\leq i\leq m$），$H_i$是$D_i$上的二元关系，$(D_i,\{H_i\})$是一棵符合本定义的树，称为根$root$的子树。

5.3.2 二叉树的抽象数据类型

类似于树，二叉树的抽象数据类型定义如下：

```
ADT BinaryTree
{
```

数据对象 D： D是具有相同特性的数据元素的集合。

数据关系 R：

若$D=\Phi$，则$R=\Phi$，称二叉树为空树；

若$D\neq\Phi$，则$R=\{H\}$ }，H是如下二元关系：

（1） 若D中存在唯一的称为根的数据元素$root$，它在关系 H 下无前驱；

（2） 若$D-\{root\}\neq\Phi$，则存在$D-\{root\}=\{D_l,D_r\}$，且$D_l\cap D_r=\Phi$；

（3） 若$D_l\neq\Phi$，则D_l中存在唯一的元素x_l，$<root,x_l>\in H$，且存在D_l上的关系$H_l\subset H$；若$D_r\neq\Phi$，则D_r中存在唯一的元素x_r，$<root,x_r>\in H$，且存在D_r上的关系$H_r\subset H$；$H=\{<root,x_l>,<root,x_r>,H_l,H_r\}$;

（4） $(D_l,\{H_l\})$是一棵符合本定义的二叉树，称为根的左子树，$(D_r,\{H_r\})$是一棵符合本定义的二叉树，称为根的右子树。

基本操作 P：

// 1. 初始化和销毁操作

InitBiTree(&T)

操作结果：构造一个空二叉树 T

DestroyBiTree(&T)

初始条件：二叉树 T 存在

操作结果：销毁二叉树 T

CreateBiTree(&T，definition)

初始条件：definition 给出二叉树 T 的定义

操作结果：按 definition 构造二叉树 T

// 2. 访问型操作

BiTreeEmpty(T)

初始条件：二叉树 T 存在

操作结果：若二叉树 T 为空树，则返回 TRUE，否则返回 FALSE

BiTreeDepth(T)

初始条件：二叉树 T 存在

操作结果：返回二叉树 T 的深度

Root(T, &e)
初始条件：二叉树 T 存在
操作结果：返回二叉树 T 的根的元素值
Value(T, cur_p, &e)
初始条件：二叉树 T 存在，cur_p 是 T 中某个结点指针
操作结果：返回 cur_p 指向的结点值 e
Parent(T, cur_p, &parent)
初始条件：二叉树 T 存在，cur_p 是 T 中某个结点指针。
操作结果：若 cur_p 非 T 的根结点，则返回它的双亲指针，否则返回值为空
LeftChild(T, cur_p, &leftChild)
初始条件：二叉树 T 存在，cur_p 是 T 中某个结点指针。
操作结果：返回 cur_p 的左孩子指针，若无左孩子，否则返回值为空
RightChild(T, cur_p, &rightChild)
初始条件：二叉树 T 存在，cur_p 是 T 中某个结点指针。
操作结果：返回 cur_p 的右孩子，若无右孩子，否则返回值为空
LeftSibling(T, cur_p, &leftSibling)
初始条件：二叉树 T 存在，cur_p 是 T 中某个结点指针
操作结果：返回 cur_p 的左兄弟，若没有左兄弟，则返回值为空
RightSibling(T, cur_p, &rightSibling)
初始条件：二叉树 T 存在，cur_p 是 T 中某个结点指针
操作结果：返回 cur_p 的右兄弟，若没有右兄弟，则返回值为空
PreOrderTraverse(T)
初始条件：二叉树 T 存在
操作结果：先序遍历 T，对每个结点访问（打印）一次且仅一次
InOrderTraverse(T)
初始条件：二叉树 T 存在
操作结果：中序遍历 T，对每个结点访问（打印）一次且仅一次
PostOrderTraverse(T)
初始条件：二叉树 T 存在
操作结果：后序遍历 T，对每个结点访问（打印）一次且仅一次
LevelOrderTraverse(T)
初始条件：二叉树 T 存在
操作结果：层次遍历 T，对每个结点访问（打印）一次且仅一次
// 3. 加工型操作
Assign(&T，&cur_p，value)
初始条件：二叉树 T 存在，cur_p 是 T 中某个结点指针
操作结果：将 cur_p 结点元素赋值为 value
InsertChild(&T, cur_p, LR, c)
初始条件：二叉树 T 存在，cur_p 指向 T 中某个结点，LR 为 0 或 1，非空二叉树 c 与 T 不相交且右子树为空
操作结果：根据 LR 为 0 或 1，插入 c 为 T 中 cur_p 指向结点的左或右子树。cur_p 所指结点的原有左或右子树则成为 c 的右子树
DeleteChild(&T, cur_p, LR)
初始条件：二叉树 T 存在，cur_p 指向 T 中某个结点，LR 为 0 或 1
操作结果：根据 LR 为 0 或 1，删除 T 中 cur_p 指向结点的左或右子树

} // ADT BinaryTree

5.3.3 二叉树的的性质

性质 1 在二叉树的第i层上至多有2^{i-1}个结点（$i \geq 1$）。

证明：利用归纳法可以证得此性质。

当$i = 1$时，只有一个根结点，显然$2^{i-1} = 2^0 = 1$是成立的；

现在假定对所有j，$1 \le j < i$，命题成立，即第j层上至多有2^{j-1}个结点，即可证明$j = i$时命题也成立。

由归纳假设：第$i-1$层上至多有2^{i-2}个结点。由于二叉树的每个结点的度至多为2，故第i层上的最大结点数为第$i-1$层上最大结点数的2倍，即$2 \times 2^{i-2} = 2^{i-1}$。得证。

性质2 深度为k的二叉树最多包含$2^k - 1$个结点（$k \ge 1$）。

证明：由性质1可见，深度为k的二叉树包含的最大结点数为：

$$\sum_{i=1}^{k}(\text{第}i\text{层的最大结点数}) = \sum_{i=1}^{k} 2^{i-1} = 2^k - 1。$$

性质3 对任意一棵二叉树T，如果其叶结点（终端结点）数为n_0，度为2的结点数为n_2，则$n_0 = n_2 + 1$。

证明：假设二叉树T中度为1的结点数为n_1，结点总数为n，则有：

$$n = n_0 + n_1 + n_2 \qquad (5.1)$$

再来看二叉树的分支数，根据5.2节中树的定义可知，包含有n个结点的树中一定存在$n-1$个分支，也就是除了根结点，每个结点都是通过一个分支“挂”在二叉树上。同时，我们知道，度为0的结点下面没有分支，度为1的结点下面有一个分支，度为2的结点下面有2个分支。因此

$$n - 1 = 0 \times n_0 + 1 \times n_1 + 2 \times n_2 \qquad (5.2)$$

根据式（5.1）与式（5.2）可得

$$n_0 = n_2 + 1 \qquad (5.3)$$

性质4 具有n个结点的完全二叉树的深度为$\lfloor \log_2 n \rfloor + 1$。

证明：假设完全二叉树的深度为k，则根据二叉树的性质2和完全二叉树的定义有

$$2^{k-1} - 1 < n \le 2^k - 1 \text{ ，也即} 2^{k-1} \le n < 2^k$$

于是$k - 1 \le \log_2 n < k$，也即$\log_2 n < k \le \log_2 n + 1$，因为$k$只能取整数，所以$k = \lfloor \log_2 n \rfloor + 1$。

性质5 如果对一棵包含有n个结点的完全二叉树的所有结点，按照层次从上到下，每层从左到右的顺序编号（如图5.5b所示），则对任一结点i（$1 \le i \le n$），有

（1） 如果$i = 1$，则结点i是二叉树的根，无父结点；如果$i > 1$，则其双亲是结点$\lfloor i/2 \rfloor$。

（2） 若$2i > n$，则结点i没有左孩子，且结点i为叶结点；否则结点i的左孩子就是$2i$。

（3） 若$2i + 1 > n$，则结点i没有右孩子；否则结点i的右孩子就是$2i + 1$。

证明：只要先证明出（2）和（3），便可由其推导出（1）。

对于$i = 1$，根据完全二叉树的定义，其左孩子结点必然是2。如果$n < 2$，即不存在2号结点，说明结点i没有左孩子；结点i的右孩子只能是3，若$n < 3$，说明3号结点不存在，此时结点i则没有右孩子；

对于$i > 1$，可以这样来讨论：

- 假设第j层的第一个结点编号为i，根据二叉树的定义以及性质2可知$i = 2^{j-1}$，则其左孩子必为第$j+1$层的第一个结点，$j+1$层的第一个结点编号应该为$2^j = 2 \times 2^{j-1} = 2i$，若$2i > n$，则没有左孩子；其右孩子必然是$j+1$层的第2个结点，编号为$2i+1$，若$2i + 1 > n$，则说明右孩子不存在。
- 由上可知，假设第j层的第二个结点编号为$i+1$，其左孩子必为第$j+1$层的第3个结点$2i + 2 = 2(i+1)$，右孩子必为第$j+1$层的第四个结点$2i + 3 = 2(i+1) + 1$。以此类推，即可得到完全二叉树中结点及其左、右孩子之间的关系。

5.3.4 二叉树的存储结构

5.3.4.1 二叉树的顺序存储

二叉树的顺序存储就是用一组连续的存储单元来存放二叉树的数据元素。首先要对树中的每个结点进行编号，编号顺序就是结点在顺序表中的存储顺序。编号的方法是：按照完全二叉树的形式，根结点的编号为 1，然后按照层次从上到下、每层从左到右的顺序对每个结点进行编号。当某结点是编号为i的结点的左孩子时，它的编号应为$2i$，当它是右孩子时编号则为$2i+1$。二叉树的顺序存储结构的类型定义如下。

二叉树的顺序存储表示：

```
#define MAX_TREE_SIZE  256                    // 二叉树的最大结点数
typedef TElem SqBiTree[MAX_TREE_SIZE];   // 根结点存储在 1 号单元
SqBiTree bt;
```

例如，图 5.5b 所示的完全二叉树的顺序存储结构如下：

1	2	3	4	5	6	7	8	9	10
A	B	C	D	E	F	G	H	I	J

而对于非完全二叉树，在采用顺序存储时，首先将其按照完全二叉树的形式补全，然后进行编号，其对应的顺序存储结构（其中#表示空）如图 5.6 所示。

1	2	3	4	5	6	7	8	9	10
A	B	C	#	D	#	E	#	#	F

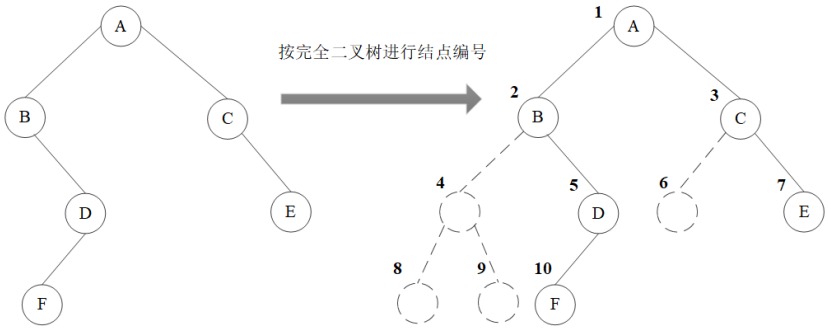


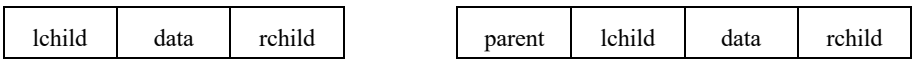
图 5.6 二叉树顺序存储编号示意图

在二叉树的顺序存储结构中，根据二叉树的性质 5 可得，若已知一个结点的编号（顺序存储位置），就能够很方便地找到其双亲和孩子结点。但该存储结构不适合结点的删除和插入，也就是说不适合结点出现动态变化的情况。此外，用顺序结构存储非完全二叉树时，顺序表中会出现很多空闲的位置。

5.3.4.2 二叉树的链式存储

二叉树的链式存储就是二叉树中的每个结点用链表中的一个链结点来存储。设计不同的结点结构可构成不同形式的链式结构。

根据二叉树的定义可知，二叉树的一个结点由一个数据元素和分别指向其左、右孩子的两个分支构成，那么用来表示二叉树结点的链结点则至少应该包含 3 个域：数据域和左、右指针域，如图 5.7a 所示。这种存储结构称为**二叉链表**，如图 5.8 所示，链表的头指针指向二叉树的根结点。有时，为了便于找到结点的双亲，还可以在结点中增加一个指向其父结点的域，如图 5.7b 所示，这种存储结构称之为**三叉链表**。

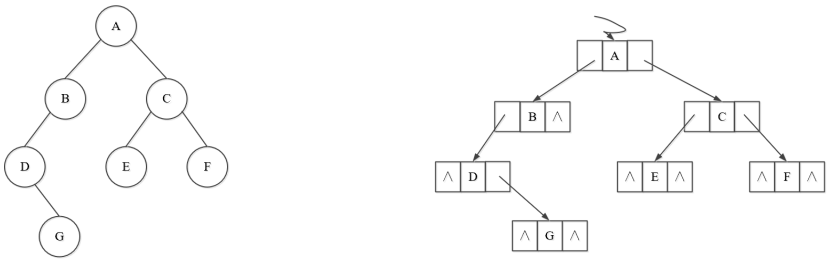


a) 二叉链表　　b) 三叉链表

图 5.7 二叉树的链式存储结构

二叉树的二叉链表存储表示：

```
typedef struct BiTNode
{
    TElemType       data;              // 数据元素
    struct BiTNode   *lchild,*rchild;  // 左、右孩子指针
}BiTNode,*BiTree;
```



(a) 二叉树　　(b) 二叉链存储结构

图 5.8 二叉树的链式存储结构

5.3.5 二叉树的遍历

二叉树的遍历是指按照一定的次序访问树中的所有结点，并且每个结点只能被访问一次的过程。它是二叉树的基本操作，也是二叉树其它常用操作的基础。

二叉树的常见遍历方法有 4 种：先序遍历、中序遍历、后序遍历以及层次遍历。

5.3.5.1 先序遍历

二叉树先序遍历的过程如下：

（1） 访问根结点；

（2） 先序遍历左子树；

（3） 先序遍历右子树。

该过程的结果表示为 rLR 型。其中 r 表示根结点，L 和 R 分别表示左、右子树，在每个子树中也是递归进行先序遍历。

例如，图 5.8 中的二叉树的先序遍历结果为：A B D G C E F，如图 5.9 所示。

先序遍历二叉树的递归算法如下：

算法 5.1 二叉树先序遍历的递归算法

```
void PreOrderTraverse( BiTree  T )
{
    if( T!=NULL )
    {
        printf("%c ",T->data);             // 访问根结点
        PreOrderTraverse( T->lchild );  // 先序遍历左子树
        PreOrderTraverse( T->rchild ); // 先序遍历右子树
    }
}// PreOrderTraverse
```

相应地，先序遍历二叉树的非递归算法如下：

算法 5.2 二叉树先序遍历的非递归算法

```
void PreOrderTraverse_NonRecursion(BiTree T)
{
    SeqStack S;
    InitStack(S);
    p = T;
    while ( p!=NULL || !StackEmpty(S) )
    {   while ( p!=NULL )          // 遍历左子树
        {   printf("%c ", p->data);
            Push( S, p );
            p = p->lchild;
        }// end while (p!=NULL)
        if ( !StackEmpty(S) )
        {   Pop(S, p);             // 回退到双亲结点
            p = p->rchild;         //  进入右子树
        }// end if
    }// end while ( p!=NULL || !StackEmpty(S) )
}// PreOrderTraverse_NonRecursion
```

可见，先序遍历二叉树的非递归算法的代码没有递归算法直观，但通过对非递归算法的阅读和调试，可以更好理解遍历路径的行走过程。

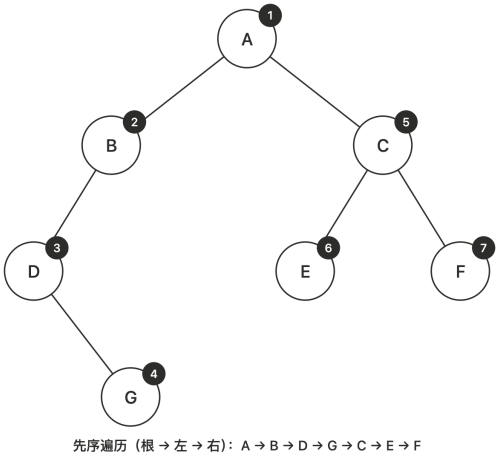


图 5.9 二叉树的先序遍历示意图

5.3.5.2 中序遍历

中序遍历的过程如下：

（1） 中序遍历左子树；

（2） 访问根结点；

（3） 中序遍历右子树。

该过程的结果表示为 LrR 型。图 5.8 中的二叉树的中序遍历结果为：D G B A E C F，如图 5.10 所示。

中序遍历二叉树的递归算法如下。

算法 5.3 二叉树中序遍历的递归算法

```
void InOrderTraverse( BiTree  T )
{
```

```
    if( T!=NULL )
    {
        InOrderTraverse( T->lchild );   // 中序遍历左子树
        printf("%c ",T->data);          // 访问根结点
        InOrderTraverse(T->rchild);     // 中序遍历右子树
    }
}// InOrderTraverse
```

相应地，中序遍历二叉树的非递归算法如下：

算法 5.4 二叉树中序遍历的非递归算法

```
void InOrderTraverse_NonRecursion(BiTree T)
{
    SeqStack S;
    InitStack(S);
    p = T;
    while ( p!=NULL || !StackEmpty(S) )
    {   while ( p!=NULL )          // 遍历左子树，向左走到头
        {   Push( S, p );
            p = p->lchild;
        }// end while (p!=NULL)
        if ( !StackEmpty(S) )
        {   Pop(S, p);             // 回退到双亲结点
            printf("%c ", p->data); // 访问结点
            p = p->rchild;         // 进入当前结点的右子树
        }// end if
    }// end while ( p!=NULL || !StackEmpty(S) )
}// InOrderTraverse_NonRecursion
```

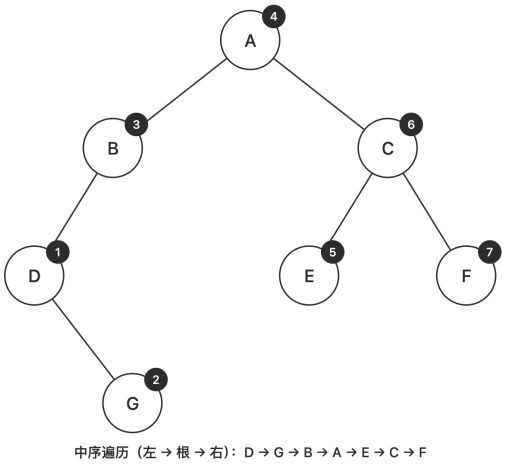


图 5.10 二叉树的中序遍历示意图

5.3.5.3 后序遍历

后序遍历的过程如下：

（1） 后序遍历左子树；

（2） 后序遍历右子树；

（3） 访问根结点。

该过程的结果表示为 LRr 型。图 5.8 中的二叉树的后序遍历结果为：G D B E F C A，如图 5.11 所示。

后序遍历二叉树的递归算法如下：

算法 5. 5 二叉树后序遍历的递归算法

```
void PostOrderTraverse(BiTNode T)
{
    if(T!=NULL)
    {
        PostOrderTraverse( T->lchild );    // 后序遍历左子树
        PostOrderTraverse( T->rchild );    // 后序遍历右子树
        printf("%c ",T->data);             // 访问根结点
    }
}// PostOrderTraverse
```

相应地，后序遍历二叉树的非递归算法如下：

算法 5. 6 二叉树后序遍历的非递归算法

```
void PostOrderTraverse_NonRecursion(BiTree T)
{
    SeqStack   S;
    ElemType x;       //修改栈中元素定义，除了放结点指针 ptr，还增加一个标记标量 tag
    InitStack(S);
    p = T;
    do
    {   while ( p!=NULL )          // 遍历左子树，向左走到头
        {   x.ptr = p;
            x.tag = 'L';                // 标记为从左子树进栈
            Push( S, x );
            p = p->lchild;
        }// end while (p!=NULL)
        while (!StackEmpty(s) && *(S.pTop-1).tag==R)
        {   Pop(S, x);
            p = x.ptr;
            printf("%c ", p->data); // 访问结点,此时 tag 为 R，表示右子树访问完毕，故访问根结点
        }
        if ( !StackEmpty(S) )
        {   *(S.pTop-1).tag='R';                // 修改栈顶结点的过栈标记
            p = *(S.pTop-1).ptr->rchild;      // 进入当前结点的右子树
        }// end if
    }while ( !StackEmpty(S) )
}// PostOrderTraverse_NonRecursion
```

以上给出的是二叉树先序、中序和后序遍历的递归算法和非递归算法。其中，非递归算法利用栈结构实现，将根结点压栈，可以很方便地通过弹栈操作从左子树经过根进入到右子树。需要注意的是，后序遍历的顺序是左子树、右子树和根。从左子树到右子树，再从右子树到根，根结点需要过栈两次才能最终回到根结点。

进一步地，针对二叉树的先序、中序和后序非递归遍历方法，也可以统一到一个框架下，如算法 5.7 所示。当 order 分别为“rLR”、“LrR”和“LRr”时，对应先序、中序和后序的非递归遍历。

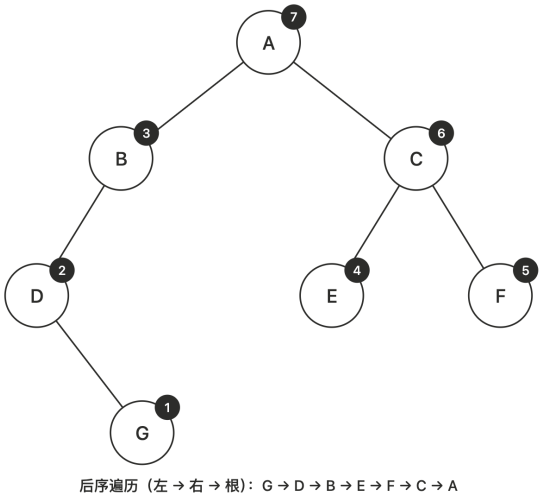


图 5.11 二叉树的后序遍历示意图

算法 5.7 二叉树遍历的非递归算法

```
void TraverseBiTree_NonRecursion (BiTree T, char order[])
{   Stack S;
    InitStack(S);
    BiTree p = T;
    int  tagIndex;

    Push(S, p, 0);
    while(!IsEmpty(S))
    {       Pop(S, p, tagIndex);
            if( tagIndex+1 < ORDER_MAX_LEN )
            {           Push(S, p, tagIndex+1);       }
            switch( order[tagIndex] )
            {    case 'L':
                        if(p->lchild)       Push(S,p->lchild,0);
                        break;
                 case 'R':
                        if(p->rchild)       Push(S,p->rchild,0);
                        break;
                 case 'r':
                        cout<<p->val<<" ";
                        break;
                 default:
                        cerr<<"invaild order!";
                        exit(-1);
            } //end switch
        }  // end while
    }  // TraverseBiTree_NonRecursion
```

5.3.5.4 层次遍历

层次遍历的过程如下：

（1） 访问根结点（第一层）；

（2） 从左到右访问第二层的所有结点；

（3） 从左到右访问第三层的所有结点，…，直至最后一层的所有结点。

图 5.8 中的二叉树的层次遍历结果为：A B C D E F G，如图 5.12 所示。

层次遍历二叉树的算法如下。

算法 5.8 二叉树层次遍历算法

```
#define MAX_SIZE   256                    // 二叉树层次遍历队列最大空间
void LevelOrderTraverse( BiTree  T  )
{
    BiTNode *p;
    SeqQueue   Q;                         // 定义队列，存放二叉树结点指针
    InitQueue( Q );                       // 初始化队列
    EnQueque(Q, T);                       // 根结点入队
    while( !QueueEmpty(Q) )               // 只要队列不为空
    {
        DeQueue(Q, p);                    // 队首结点出队
        printf("%c ",p->data);            // 访问队首结点
        if(p->lchild!=NULL)               // 若当前访问的结点有左孩子，则其左孩子入队
        {
            EnQueue(Q, p->lchild);
         }
         if(p->rchild!=NULL)              // 若当前访问的结点有右孩子，则其右孩子入队
        {
            EnQueue(Q, p->rchild);
        }
    } // end while ( !QueueEmpty(Q) )
}// LevelOrderTraverse
```

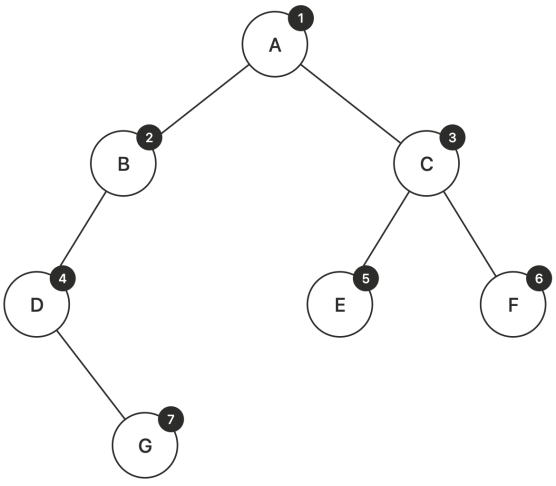


图 5.12 二叉树的层次遍历示意图

5.3.6 二叉树遍历算法的应用举例

下面以二叉链表存储结构为例，介绍几个常见的二叉树基本操作运算的实现。

5.3.6.1 创建二叉树

这里介绍一个按照类似于先序序列创建二叉树的过程。对图 5.8 所示的二叉树，按照如下的先序顺序读入字符：

A B D Φ G Φ Φ Φ C E Φ Φ F Φ Φ

其中Φ表示空格字符（对应在先序遍历中遇到的空指针），可以根据先序遍历的顺序创建对应的二叉链表。也就是说，首先创建根结点，然后先序创建左分支，接下来先序创建右分支，最后返回根结点指针，创建完成。对应的算法如下。

算法 5.9 基于先序序列的二叉树创建算法

```
Status CreateBiTree( BiTNode &T )
{
    scanf("%c",&ch);                                    // 读入一个字符
    if ( ch==' ' )  T = NULL;                           // 空格字符代表空指针
    else
    {
        T = (BiTNode *)malloc(sizeof(BiTNode));         // 创建根结点
        if ( !T )    exit( OVERFLOW );
        T->data = ch;                                   // 为根结点数值域赋值
        CreateBiTree( T->lchild );                      // 先序创建左分支
        CreateBiTree( T->rchild );                      // 先序创建右分支
    }
    return OK;
}// CreateBiTree
```

5.3.6.2 查找二叉树中的结点

根据先序遍历的顺序在二叉树中查找值为 x 的结点，找到后返回其结点指针，否则返回 NULL。算法步骤如下：

（1） 先将根结点的值与 x 做比较，若相等，则返回结点指针，查找结束；

（2） 在左分支中先序查找 x，如果找到，则返回相应的结点指针，查找结束；

（3） 在右分支中先序查找 x，如果找到，则返回相应的结点指针，查找结束。

算法 5.10 基于先序顺序的二叉树结点查找算法

```
BiTNode *FindNode ( BiTree T,  TElemType x )
{
    BiTNode *p=NULL;
    if( T==NULL )  return NULL;
    else if( T->data==x )  return T; // 如果访问的当前结点值等于 x，则返回当前结点指针，查找结束
    else
    {
        p = FindNode( T->lchild,x ); //在左分支中先序查找
        if( p==NULL )
                return FindNode( T->rchild,x ); //左分支中没有找到，继续先序查找右分支，并返回查找结果
        else
                return p; // 如果在左分支中找到 x，则返回该结点指针，查找结束
    } // end else
}// FindNode
```

5.3.6.3 计算二叉树的高度

为了得到二叉树的高度，首先需要计算其左、右子树的高度，然后取左、右子树高度的最大值，再加上根结点这一层的高度，就可得到整个二叉树的高度。 左、右子树的高度，可通过递归求解。显然，该描述与后续遍历的步骤一致。

因此，可按照后序遍历的顺序来计算二叉树的高度。计算步骤如下：

（1） 计算左子树的高度；

（2） 计算右子树的高度；

（3） 根据对树的高度的定义，取上述二者中的最大值并加 1，即为二叉树在根结点处的高度。

具体代码如下。

算法 5.11 基于后序顺序的二叉树高度计算算法

```
int BiTreeDepth(BiTree T)
{
    int leftdepth, rightdepth;
    if( T==NULL )  return 0;                                    // 空树的高度为 0
    else
    {
        leftdepth   = BiTreeDepth(T->lchild);                   // 求左子树的高度
        rightdepth = BiTreeDepth(T->rchild);                    // 求右子树的高度
        return (leftdepth>rightdepth) ? (leftdepth+1) : (rightdepth+1);  // 最终得到二叉树的高度
    }
}// BiTreeDepth
```

5.3.6.4 输出二叉树

将二叉树利用图 5.3c 的括号表示法进行输出显示，其过程是基于先序的顺序，即最先显示根结点，然后先序显示左分支，最后先序显示右分支。步骤如下：

（1） 对于非空的二叉树，首先输出根结点；

（2） 如果根结点有左孩子或者右孩子，则输出一个左括号“（”；

（3） 先序输出其左分支；

（4） 输出一个“，”作为分隔左、右分支的分隔符；

（5） 先序输出右分支；

（6） 输出右括号“）”，对二叉树的输出显示结束。

算法 5.12 基于先序顺序的二叉树显示递归算法

```
void ShowBiTree( BiTree T )
{
    if( T )
    {
        printf("%c",T->data);
        if ( T->lchild || T->rchild )
        {
            printf("(");
            ShowBiTree(T->lchild);              // 先序输出左子树
            printf(",");
            ShowBiTree(T->rchild);              //  先序输出右子树
            printf(")");
        } // end if ( T->lchild || T->rchild )
```

```
    } // end if ( T )
}// ShowBiTree
```

将上述算法作用于图 5.8 所示的二叉树，输出结果为 A(B(D(,G),),C(E,F))。

5.3.6.5 销毁二叉树

在销毁一个二叉树时，为了防止结点指针丢失，一定要按照后序遍历的顺序进行，即先后序销毁左分支，再后序销毁右分支，最后销毁根结点，最终二叉树为空。注意，销毁不能按照先序的顺序，因为如果是先序销毁，那么最先销毁的是根结点，而一旦根结点被销毁，就将无法找到其左、右分支，也就无法对其左、右分支进行销毁。同理，销毁也不能按照中序的顺序进行。

算法 5.13 基于后序顺序的二叉树销毁算法

```
BiTree DestroyBiTree(BiTree T)
{
   if( T )
   {
      T->lchild=DestroyBiTree(T->lchild); // 销毁左子树
      T->rchild=DestroyBiTree(T->rchild); // 销毁右子树
      free(T);                            // 销毁根结点
      return NULL;
   }  // end if ( T )
}// DestroyBiTree
```

5.3.6.6 判断二叉树是否为完全二叉树

根据定义，完全二叉树应满足以下条件：

（1） 叶结点只能出现在树的最后两层。

（2） 度为 1 的结点最多只能有一个，而且这个结点只能是有左孩子，没有右孩子。

（3） 如果分别用n_2、n_1和n_0表示完全二叉树中度为 2、度为 1 以及度为 0 的结点个数，那么它们的取值只会是这三种情况中的一种，即$n_2 = 0$, $n_1 = 0$, $n_0 = 1$；$n_2 = 0$, $n_1 = 1$, $n_0 = 1$；$n_2 \neq 0$, $n_1 = 1$或 0, $n_0 \neq 0$。

（4） 从层次遍历的顺序来说，如果有度为 2 的结点，那么会最先遍历这些结点，然后会遍历度为 1 的结点（如果有），最后才会遍历叶结点。

如果未同时满足以上条件，就说明该二叉树不是完全二叉树。而这些条件是否被满足可以在层次遍历的过程中进行判断。

算法 5.14 基于层次遍历的判断二叉树是否为完全二叉树的算法

```
#define MAX_SIZE   256              // 二叉树层次遍历队列最大结点数
int IsComplete( BiTree T )          // 判断二叉树是否是完全二叉树，是返回 TRUE，否则返回 FALSE
{
   BiTNode *p;
   SeqQueue Q;                      // // Q 队列，用来对树做层次遍历
   int n1=0;                        // 若有度不大于 1 的结点被访问，则 n1 设置为 1，初始值为 0
   InitQueue( Q );
   EnQueue( Q, T) ;                 // 根结点首先入队
   while( !QueueEmpty(Q))           // 只要队列非空则进行如下的操作
   {
      DeQueue(Q, p);                // 访问队头结点
      if( !p->lchild && !p->rchild )  // 如果当前被访问的是叶结点
         n1=1;                      //  则标识符 n1 设置为 1
```

```
        else if( !p->lchild && p->rchild )      // 如果当前被访问的是没有左分支只有右分支的结点
            return FALSE;                       // 不符合完全二叉树的条件，返回 FALSE
        else if( p->lchild && !p->rchild )      // 如果当前被访问的是只有左分支没有右分支的结点
        {
            if( n1!=0 )                         // 如果此前已经有度不大于 1 的结点被访问过
                return FALSE;                   // 不符合完全二叉树的条件，返回 FALSE
            else                                // 如果此前没有度不大于 1 的结点被访问过
            {
                n1 = 1;                         // 则标识符 n1 设置为 1
                EnQueue(Q, p->lchild);          // 当前结点的左孩子入队
            } //end else
        } // end if if( p->lchild && !p->rchild )
        else                                    // 如果当前被访问的是度等于 2 的结点
        {
            if(n1)                              // 如果此前已经有度不大于 1 的结点被访问过
                return FALSE;                   // 不符合完全二叉树的条件，返回 FALSE
            EnQueue(Q, p->lchild);              // 当前结点的左孩子入队
            EnQueue(Q, p->rchild);              // 当前结点的右孩子入队
        } //end else
    } // end while
    return TRUE;                                // 可以正常遍历完二叉树，符合完全二叉树条件，返回 TRUE
}// IsComplete
```

5.3.6.7 线索二叉树

从二叉树的链式存储结构可知，若当前结点无左子树，则其左孩子指针域为空；若当前结点无右子树，则其右孩子指针域为空。能否将这些空闲指针利用起来呢？例如在二叉树遍历时，可将空闲的左孩子指针指向其前驱结点，空闲的右孩子指针指向其后继结点。再次遍历二叉树时，可以直接通过指向后继结点的“线索”指针直接访问，从而避免递归或栈的使用。为此，人们提出了线索二叉树，如图 5.13 所示。

为了区分当前指针是指向子树还是指向线索，对二叉树的链式存储结构稍作改造，增加两个指针的标志域。

线索二叉树的相应存储结构定义如下：

```
typedef enum PointerTag {Link, Thread}; // Link==0 表示指针；Thread==1 表示线索
typedef struct BiThrNode
{
    TElemType       data;               // 数据元素
    struct BiThrNode  *lchild,*rchild;  // 左、右孩子指针
    PointerTag      LTag, RTag;         // 左、右标志
} BiThrNode, *BiThrTree;
```

探讨中序线索二叉树的建立过程。在第一遍中序遍历过程中修改各个结点的空闲指针指向前驱和后继结点，需要增加一个 pre 指针变量指向之前访问的结点。当前结点的左孩子指针为空时，则令 p->lchild=pre 使其指向前驱结点。由于无法提前预知后继结点，因此无法直接修改当前结点 p->rchild 值。但是当访问到后继结点时，原来的结点变成为 pre 指针指向的前驱结点，这时可令 pre->rchild=p，从而实现前驱结点的右孩子指针指向当前结点，也即前驱结点的右孩子指针指向其后继结点。为此，同一个结点的左、右孩子指针是在不同步骤里实现线索化，这是一个巧妙的实现方式。同时，由于二叉树根结点的左、右孩子指针已指向真实的左、右子树，因此需增加一个线索头结点 Thrt，并令其指向二叉树的根结点，实现双向链表的闭环。

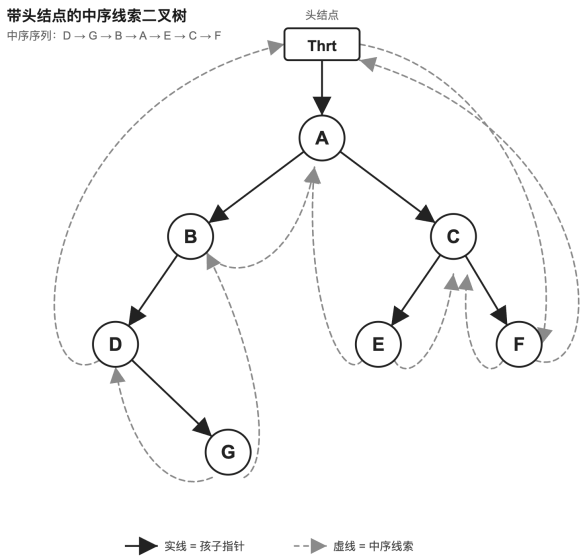


图 5.13 中序线索二叉树示意图

相应地，中序线索二叉树建立的算法如下。

算法 5.15 中序线索二叉树建立算法

```
Status InOrderThreading( BiThrTree &Thrt, BiThrTree T )
{     // 中序线索化二叉树 T，Thrt 指向头结点
      if ( ! (Thrt = (BiThrTree)malloc(sizeof(BiThrNode))) ) exit(OVERFLOW);
      Thrt->LTag = Link; Thrt->RTag = Thread;    // 建立线索头结点
      Thrt->rchild = Thrt;                       // 右指针回指
      if ( !T) Thrt->lchild = Thrt;              // 若二叉树空，则左指针回指
      else
      {    Thrt->lchild = T;     pre = Thrt;
           InThreading( T );                     // 中序遍历进行中序线索化
           pre->rchild = Thrt; pre->RTag = Thread;  // 最后一个结点线索化
           Thrt->rchild = pre;
      }
      return OK;
} // InOrderThreading
```

算法 5.16 InThreading 算法

```
void  InThreading(  BiThrTree p )
{
      if ( p )
      {    InThreading( p->lchild );             // 左子树线索化
           if( ! p->lchild) { p->LTag = Thread; p->lchild = pre; }       // 前驱线索
           if( ! pre->rchild) { pre->RTag = Thread; pre->rchild = p; }  // 后继线索
           pre = p;
           InThreading( p->rchild );             // 右子树线索化
      }
} // InThreading
```

当中序线索化完成后，后续对二叉树再中序遍历时，过程就较为简单了。相应的遍历算法如下，无需栈和递归实现。

算法 5.17 对中序线索二叉树的中序遍历算法

```
Status  InOrderTraverse_Thr(  BiThrTree T )
```

```
{    // T 指向中序线索二叉树的头结点，其 lchild 指向二叉树根结点
     p = T->lchild;
     while ( p != T )
     {     while ( p->LTag ==  Link )  p = p->lchild;          // 一直走到最左子树
           printf("%c ",p->data);
           while( p->RTag == Thread  && p->rchild != T )  // 根据线索指针访问后继结点
           {     p = p->rchild;
                 printf("%c ",p->data);
           }
           p = p->rchild;                                          // 进入右子树
     } // end while (p != T)
} // InOrderTraverse_Thr
```

采用同样的思想，也可将所有的叶子结点链接起来，形成一个叶子结点的双向链表，从而针对叶子结点的应用中可以实现对叶子结点的直接访问。

5.4 霍夫曼树

霍夫曼（Huffman）树，又叫作最优树，是一种带权路径长度最小的树，有着广泛的应用。

5.4.1 霍夫曼树的定义

在许多应用中，常常为树中的叶结点赋予一个有着某种意义的值，并称此数值为该叶结点的**权值**。从根结点到叶结点之间的路径长度与该叶结点的权值的乘积叫作该叶结点的**带权路径长度**。树中所有叶结点的带权路径长度之和称为**树的带权路径长度**，通常记为

$$WPL = \sum_{i=1}^{n} w_i l_i \qquad (5.4)$$

其中n表示叶结点的数量，w_i表示第i个叶结点的权值，l_i表示根结点到第i个叶结点的路径长度。在n个带权叶结点构成的所有二叉树中，带权路径长度WPL最小的二叉树称为**霍夫曼树**（或最优二叉树）。

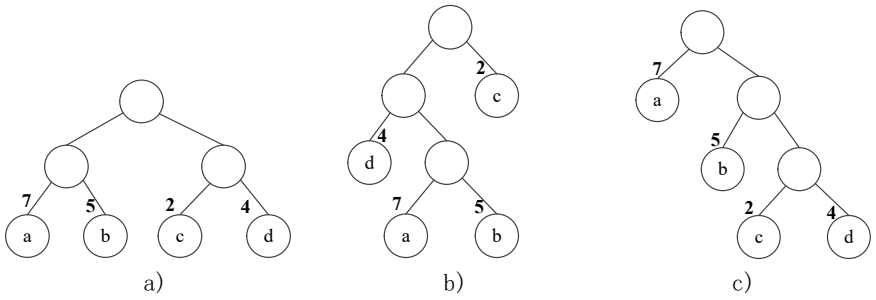


图 5.14 具有不同带权路径长度的二叉树

例如，图 5.14 中所示的 3 棵二叉树都有 4 个叶结点 a、b、c 和 d，分别具有 7、5、2、4 的权值。这 3 棵二叉树的WPL分别是

图 5.14a：$WPL = 7 \times 2 + 5 \times 2 + 2 \times 2 + 4 \times 2 = 36$

图 5.14b：$WPL = 7 \times 3 + 5 \times 3 + 2 \times 1 + 4 \times 2 = 46$

图 5.14c：$WPL = 7 \times 1 + 5 \times 2 + 2 \times 3 + 4 \times 3 = 35$

其中第 3 棵树的带权路径长度WPL最小。可以验证，其带权路径长度在所有包含 4 个叶结点且权值分别是 7、5、2、4 的二叉树中最小，因此它就是霍夫曼树。

在解决某些问题时，利用霍夫曼树可以得到最佳算法。例如，假定现有 5 个有序的数组 A、B、C、D 和 E，它们包含的数据元素的个数分别为 200、40、160、360、100，现在需要将这 5 个数组进行归并，最后得到一个整体有序的数组。那么如何做归并可以达到最高的效率呢？

其实这个问题并不复杂，只要将这 5 个有序数组两两做归并，经过 4 次归并，即可得到一个整体有序的序列。问题是，如何以最高的效率完成这 4 次归并呢？

根据第 2 章中所介绍的线性表的性质可知，若有两个有序数组 X 和 Y，其包含的数据元素个数分别为m和n，将 X 和 Y 做归并的时间复杂度是$O(m+n)$，最坏情况下需要做$m+n-1$次比较。如果要高效地完成上述 4 次归并，意味着应该以最少的比较次数完成这 4 次归并。由于每次都是将两个数组归并成为一个新的有序数组，因此，这个归并的过程可以用二叉树来表示。如图 5.15a 所示的归并过程如下：

（1） 将 A 与 B 归并为 S1；

（2） 将 C 和 D 归并为 S2；

（3） 将 S2 与 E 归并为 S3；

（4） 将 S1 与 S3 归并为 S4，S4 即为一个整体有序的数组，一共包含 860 个数据元素。

在这个过程中，图 5.15a 中所示二叉树的WPL即表示了这个归并过程中最坏情况下一共进行的比较次数（$m+n-1$中的常数项-1 忽略不计）。

同理，5.15b 所示的归并过程如下：

（1） 将 B 与 E 归并为 S1；

（2） 将 C 和 S1 归并为 S2；

（3） 将 S2 与 A 归并为 S3；

（4） 将 S3 与 D 归并为 S4，S4 即为一个整体有序的数组，一共包含 860 个数据元素。

我们可以计算一下这两种归并策略分别在最坏情况下需要的比较次数。

图 5.15a：$WPL = 200\times 2 + 40\times 2 + 160\times 3 + 360\times 3 + 100\times 2 = 2240$

图 5.15b：$WPL = 200\times 2 + 40\times 4 + 160\times 3 + 360\times 1 + 100\times 4 = 1800$

可以证明图 5.15b 是一个最优二叉树，即霍夫曼树，因此利用该策略进行归并操作是最佳的。

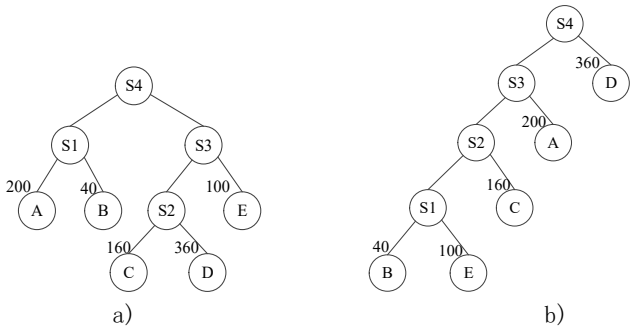


图 5.15 归并过程示意

5.4.2 霍夫曼树的构造

如何构造一个霍夫曼树呢？也就是给定n_0个权值，如何构造一个包含n_0个叶结点的二叉树，使其带权的路径长度WPL最小呢？霍夫曼最早给出了一个带有一般规律的算法，称为霍夫曼算法，步骤如下：

（1） 现有n_0个权值（$w_1, w_2, \ldots, w_{n_0}$），每个权值对应一个结点，这些结点构成一个森林$F = \{T_1, T_2, \ldots, T_{n_0}\}$，森林中的每棵树$T_i$（$1 \le i \le n_0$）都是二叉树，且都仅包含一个带权值$w_i$的根结点，左右子树均为空。

（2） 在森林中选取根结点权值最小的两棵树T_i和T_j进行合并，创建出一棵新的二叉树。新的二叉树根结点的权值为T_i和T_j两棵树根结点权值之和，新二叉树的左、右分支分别为T_i和T_j。在森林中用该新二叉树替换掉T_i和T_j，此时森林中还剩余$n_0 - 1$棵二叉树。

（3） 重复上述步骤直到森林中只剩下一棵二叉树为止，这棵二叉树便是霍夫曼树。注意，根据上述步骤创建的霍夫曼树是一棵没有度为 1 结点的二叉树。

例如有 5，9，11，15，17，19，24 七个权值，图 5.16 给出了根据这七个权值创建霍夫曼树的过程，其中 5.16g 就是最终得到的霍夫曼树，其带权路径长度$WPL = 271$。

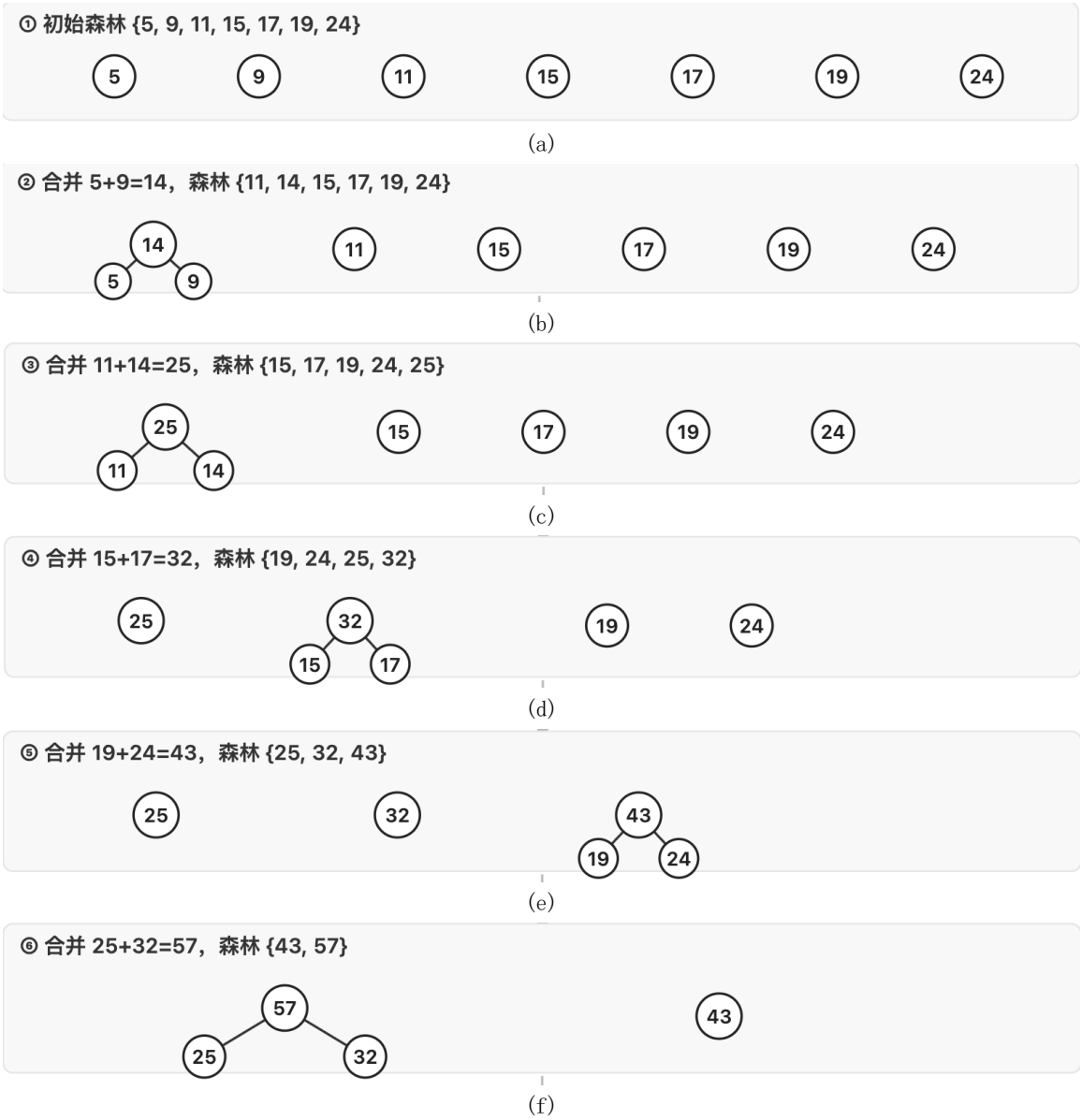
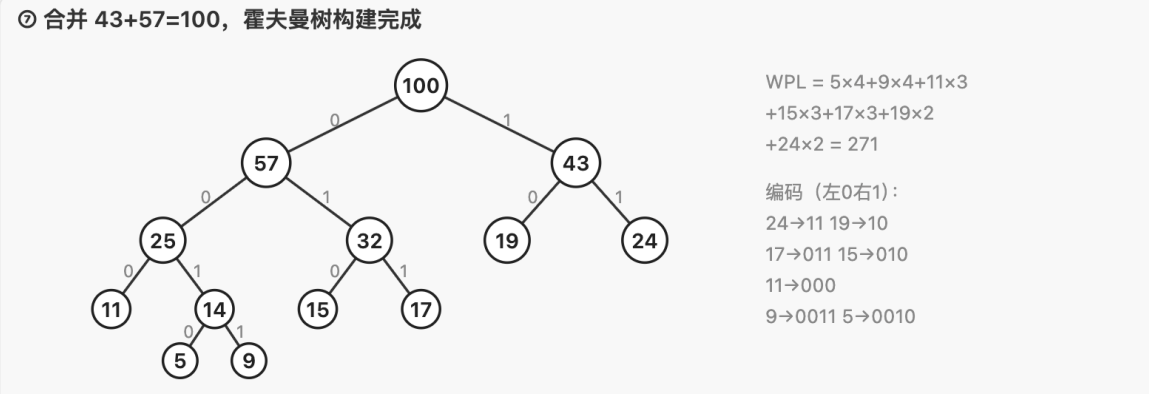


图 5.16 构造霍夫曼树的过程



(g)

图 5.16 构造霍夫曼树的过程(续)

5.4.3 霍夫曼编码

霍夫曼树可以应用于文件压缩。

标准的 ASCII 字符集由大约 100 个“可打印”字符组成，这些字符在计算机中用$\lceil \log_2 100 \rceil = 7$位的二进制编码来表示。但 7 位编码共可以表示 128 个字符，因此 ASCII 字符集还可以再添加一些其它“不可打印”的符号。重点在于，如果字符集的大小是M，那么这个字符集中每个符号的标准编码就需要$\lceil \log_2 M \rceil$位（bit）。

在现实中，一个文件可能非常大。如果在带宽有限的网络上传输这些文件，人们就会希望减少文件的大小。同时每台机器上的存储空间也都是非常珍贵的，因此人们希望提供一种更有效的编码机制以降低文件所占用的总比特数。由于在一个文件中，使用频率最高和使用频率最低的字符之间通常存在很大的差别，因此，一种简单的策略就是让编码的长度随字符的使用频率变化，以保证经常出现的符号编码要短。霍夫曼树就可以提供一种最佳的编码方案，下面将对其举例说明。

设有一个文件，它只包含有 a、e、i、s、t 五种字符，如果要对这五个符号进行编码，那么每个符号需用$\lceil \log_2 5 \rceil = 3$位。这些符号在文件中出现的频次及其在文件中占用的总位数如表 5.1 所示。

表 5.1 标准编码方案

字符	标准编码	出现频次	总位数
a	000	10	30
e	001	15	45
i	010	12	36
s	011	3	9
t	100	4	12
总计			132

接下来可以利用霍夫曼树对该文件中的字符进行重新编码，以达到数据压缩的目的。现以这五种字符作为叶子结点，以它们各自在文件中出现的频次为其权值，创建霍夫曼树，如图 5.17 所示。规定霍夫曼树中的左分支为 0，右分支为 1（反之也可以），则从根结点到某叶结点所经过的分支对应的 0 或者 1 组成的序列就是该叶结点对应的编码，这样的编码就叫做霍夫曼编码。按照这样的编码策略，可以得到最终的编码结果，如表 5.2 所示。可以发现，文件整体所占位数下降了，即文件得到了压缩。

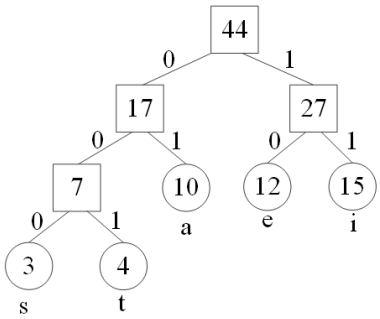


图 5.17 字符的霍夫曼编码

由上述的编码过程，可以发现：

（1） 霍夫曼编码的实质就是让使用频率越高的符号采用的编码越短。

（2） 对于一个要压缩的文件，其霍夫曼编码并不唯一。原因有：首先，在创建霍夫曼树的过程中，当合并两个二叉树时，没有规定必须由哪个作为新树的左分支、哪个作为右分支；其次，在对霍夫曼树的左、右分支做 0 或者 1 的规定时，既可以左为 0、右为 1，也可以左为 1、右为 0。因此，对于一个给定的带权值的叶结点集合，其霍夫曼树并不是唯一的，相应地，霍夫曼编码也不是唯一的。

（3） 霍夫曼编码都是前缀码。由于霍夫曼编码长度不等，所以为了译码方便，任何一个短码都不可能是其它长码的前缀。

（4） 如果一个文件中所有的符号都以相同的频次出现，那么利用霍夫曼编码将无法达到压缩的目的。

表 5.2 霍夫曼编码方案

字符	霍夫曼编码	出现频次	总位数
a	01	10	20
e	10	15	30
i	11	12	24
s	000	3	9
t	001	4	12
总计			95

5.4.4 霍夫曼树和霍夫曼编码的算法实现

根据霍夫曼树的创建过程可知，树中没有度为 1 的结点。因此包含有n个叶结点的霍夫曼树一共具有n个叶结点和$n-1$个度为 2 的中间结点，共计$2n-1$个结点，这些结点可以存放在一个长度为$2n-1$的一维数组中。那么如何定义这些结点的结构呢？由于在创建霍夫曼树以及编码的过程

中，对于每个结点，不仅需要知道其双亲结点的信息，也需要知道其孩子结点的信息，因此，设定如下霍夫曼树结点的存储结构：

```
typedef struct
{
    unsigned int weight;
    unsigned int parent, lchild, rchild;
}HTNode,*HufTree;
```

以表 5.1 中给定的字符集为例，针对这个字符集，叶结点数量$n = 5$，那么相应的霍夫曼树一共应包含$2n - 1 = 9$个结点。因此表示霍夫曼树的一维数组的表长可以设定为 10（0 号位置不用，从 1 号位置开始存放结点数据），其初始状态如图 5.18a 所示。根据前述的创建过程，最终得到的霍夫曼树的结果如图 5.18b 所示。

	weight	parent	lchild	rchild
1	10	0	0	0
2	15	0	0	0
3	12	0	0	0
4	3	0	0	0
5	4	0	0	0
6	-	0	0	0
7	-	0	0	0
8	-	0	0	0
9	-	0	0	0

a）初始化状态

	weight	parent	lchild	rchild
1	10	7	0	0
2	15	8	0	0
3	12	8	0	0
4	3	6	0	0
5	4	6	0	0
6	7	7	4	5
7	17	9	6	1
8	27	9	3	2
9	44	0	7	8

b）终止状态

图 5. 18 霍夫曼树的存储结构

创建霍夫曼树以及获取霍夫曼编码的算法如下：

算法 5. 18 霍夫曼树创建和编码

```
void HufCode( int n, int *w, HufTree HT, char **HC )
{
    // n 表示叶结点数量
    // w 是用来存放叶结点权值的一维数组名
    // HT 用来表示存储霍夫曼树的一维数组名，下面将动态地为其分配空间，分配的空间长度是 2n
    // HC 是存储霍夫曼编码的码表，由于编码长度不一样，因此每个编码的码表长度应动态分配

    if( n<=1 ) return;          // 只有一个结点，直接返回
    // 0 号单元不用，因此虽然树中只有 2n-1 个结点，但是表长要设定为 2n
    HT=(HufTree)malloc(2*n*sizeof(HTNode));
    initHT( HT );               // 对表示霍夫曼树的一维数组进行初始化

    // 第 1 步：创建霍夫曼树
    for( i=n+1; i<2*n; i++ )
    {
        // 在数组 HT 中,从 1 号到 i-1 号区间中选取两个 parent 为 0，且权值 weight 最小的两个结点
        // s1 和 s2 分别表示这两个结点在 HT 中的编号
        select_two_small( HT, i-1, s1, s2 );
        HT[s1].parent = HT[s2].parent = i;
        HT[i].lchild = s1;
        HT[i].rchild = s2;
        HT[i].weight = HT[s1].weight+HT[s2].weight;
    } // end for
```

```
    // 第 2 步：从叶结点到根结点逆向获取每个叶结点的霍夫曼编码
    HC = (char **)malloc(n*sizeof(char *)); // 动态分配 n 个编码的行指针向量
    temp = (char *)malloc(n*sizeof(char));  // 分配一个临时空间用来存放当前获取的编码
    for( i=1; i<=n; i++)
    {
        start=0;
        for(c=i, f=HT[i].parent; f!=0; c=f, f=HT[f].parent ) //从叶子结点到根结点逆向获取编码的每一位
        {
            if( HT[f].lchild==c )
                temp[start++] = '0';  // 左分支为 0
            else
                temp[start++] = '1';  // 右分支为 1
        } // end for
        temp[start++] = '\0';           // 为编码设置结束符
        reverse(temp);          // 将编码逆置。由于是逆向获取的编码，因此逆置后才是真正的霍夫曼编码
        HC[i]=(char *)malloc(start*sizeof(char)); // 为第 i 个字符的编码动态分配空间
        strcpy( HC[i],temp ); // 从临时空间中将编码复制到码表 HC 中
    } // end for (i=1; i<=n; i++)
    free(temp); // 编码结束，释放临时空间
    return ;
}// HufCode
```

解码就是分解码流中的字符串：从霍夫曼树的根结点出发，按字符“0”或者“1”寻找左孩子或者右孩子，直至找到叶子结点。这个子串对应的就是该叶结点相应的字符，具体算法读者可以自己试着完成。

5.5 树和森林

5.5.1 树的存储结构

树的存储结构有很多形式。这里，我们介绍一种树的二叉链存储结构。链表中结点的两个链域分别指向该结点的第一个孩子和下一个兄弟，将其分别命名为 firstchild 和 nextbrother 域。相对于二叉树形式，我们也常称其为“左孩子、右兄弟”表示法。

树的二叉链存储表示：

```
typedef struct TreeNode
{
    TElemType       data;
    struct TreeNode    *firstchild,* nextbrother;  // 孩子-兄弟指针
}TreeNode,*Tree;
```

由此可见，对于一棵给定的树，可以通过这种二叉链的表示方式找到唯一的一棵二叉树与之对应，从物理结构来看，它们的二叉链表是相同的，只是解释不同而已，如图 5.19 所示。这样一来，对树的各种操作都可以转换为对相应的二叉树进行操作。

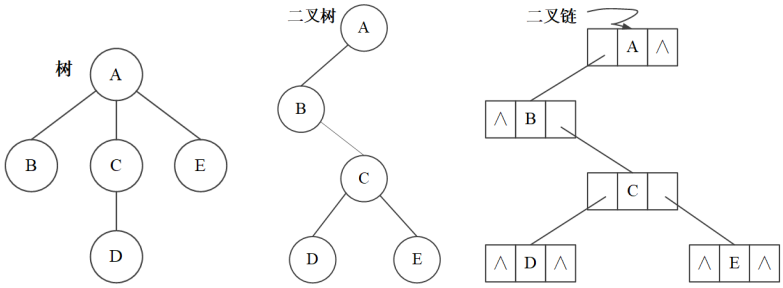


图 5.19 树与二叉树的对应关系示例

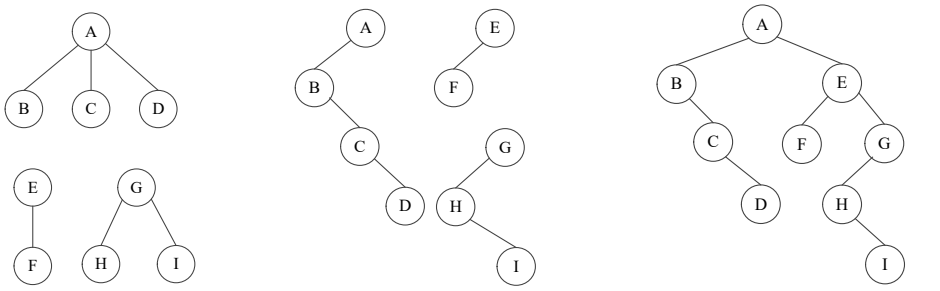
5.5.2 森林的存储结构

1. 森林转换为二叉树

森林是树的集合。如果$F=\{T_1,T_2,\ldots,T_m\}$是森林，森林中的每棵树T_i（$1\le i\le m$）都可转换成一棵二叉树。根据树的二叉链的定义可知，任何一棵树所对应的二叉树，其右子树必然为空。因此，森林F也可以按照如下规则转换成一棵二叉树$B=(root,LB,RB)$：

（1） 若F为空，即$m=0$，则B为空树；

（2） 若F非空，即$m\neq 0$，则B的根$root$是森林中第一棵树T_1的根；B的左子树LB即是T_1对应的二叉树的左子树；B的右子树RB是森林$\{T_2,T_3,\ldots,T_m\}$转换而来的二叉树。图 5.20 是森林转换为一棵二叉树的示意图。



a) 森林　　b) 森林中的树分别转换成二叉树　　c) 森林转换成一棵二叉树

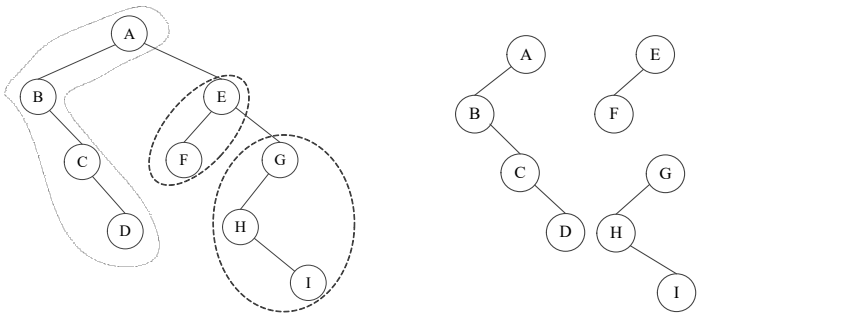
图 5.20 森林转换成二叉树

2. 二叉树转换为森林

如果$B=(root,LB,RB)$是一棵二叉树，则可以按照如下规则将其转换为森林$F=\{T_1,T_2,\ldots,T_m\}$：

（1） 若B为空，则F为空；

（2） 若B非空，则F中的第一棵树T_1的根即为二叉树 B 的根$root$；T_1的左子树即为B的左子树LB，T_1的右子树为空；B的右子树RB可以转换成除T_1以外的森林$\{T_2, T_3, \ldots, T_m\}$。图 5.21 是将二叉树还原为森林的示意图。



a) 森林　　b) 还原出的由三棵树组成的森林

图 5.21 二叉树还原为森林

5.5.3 树和森林的遍历

由上述树和森林与二叉树的转换可以看出，对树和森林的操作都可转换为对二叉树的操作来实现。

1. 树的遍历

根据树的结构特点可以引出三种顺序的遍历方法，即先根遍历、后根遍历以及层次遍历。

树的**先根遍历**是指先访问根结点，然后按照从左到右的顺序依次先根遍历树的各个子树。若对图 5.14 中所示的树进行先根遍历，结果应该是：A B C D E。

树的**后根遍历**是指首先按照从左到右的顺序依次后根遍历树的各个子树，最后访问根结点。若对图 5.14 中所示的树进行后根遍历，结果应该是：B D C E A。

树的**层次遍历**与二叉树的层次遍历类似，从根结点所在层开始，一层一层遍历结点，直到访问完最后一层的所有结点。若对图 5.14 中所示的树进行层次遍历，结果应该是：A B C E D。

树的先根遍历可借用二叉树的先序遍历算法实现；树的后根遍历可借助二叉树的中序遍历算法实现。

2. 森林的遍历

按照森林与树的相互递归的定义，可以推导出森林的两种遍历算法，即先序遍历和中序遍历。

若森林非空，则森林的**先序遍历**过程为：

（1） 访问森林中第一棵树的根结点；

（2） 先序遍历第一棵树的根结点的子树；

（3） 先序遍历除去第一棵树之后剩余的树构成的森林。

若对图 5.15 中所示的森林进行先序遍历，其结果为：

A B C D E F G H I

若森林非空，则森林的**中序遍历**过程为：

（1） 中序遍历森林中第一棵树的根结点的子树；

（2） 访问森林中第一棵树的根结点；

（3） 中序遍历除去第一棵树之后剩余的树构成的森林。

若对图 5.15 中所示的森林进行中序遍历，其结果为：

B C D A F E H I G

根据森林与二叉树的转换规则可知，在将森林转换成一棵二叉树时，其第一棵树的子树转换为二叉树的左子树，剩余的森林转换为二叉树的右子树。因此上述森林的先序遍历和中序遍历分别对应二叉树的先序遍历和中序遍历。

5.5.4 树与森林的应用举例

下面以树的二叉链存储结构为例，介绍几个常见的树的基本操作运算的实现。

5.5.4.2 创建树

假设以二元组（F,C）的形式自上而下、自左而右依次输入树的各结点链接关系，建立树的二叉链存储结构。以图 5.14 中的树为例，对应的输入序列为（‘#’，‘A’）、（‘A’，‘B’）、（‘A’，‘C’）、（‘A’，‘E’）、（‘C’，‘D’）。可见，算法中需要一个队列保存已建好结点的指针。

具体代码如下。

算法 5.19 创建树的二叉链存储

```
void CreateTree(Tree &T )
{
    T = NULL;
    for ( scanf(&fa, &ch); ch!=' '; scanf(&fa, &ch) )
    {
        p = GetTreeNode(ch);          // 创建结点
        EnQueue( Q, p);               // 结点指针入队
        if ( fa =='#')  T = p;        // 所建结点为跟结点
        else                          // 非根结点情况
        {
            GetHead( Q, s);           // 取队列队头元素值
            while ( s->data != fa )   //  查询双亲结点
            {    DeQueue(Q, s);  GetHead(Q, s);
            } // end while
            if ( ! (s->firstchild) )
            {    s->firstchild = p; r = p;  // 链接第一个孩子结点
            }
            else
            {    r->nextbrother = p; r=p;   // 链接其他结点
            }
        }// end else
    } // end for
}  // CreateTree
```

5.5.4.2 计算树的高度

与计算二叉树的高度类似，可用树的后根遍历方法得到树的高度。但需注意，与二叉树不同的是树的左孩子与右兄弟结点不在同一层次上。首先计算其左孩子、右兄弟的高度，然后取左孩子高度加 1 和右兄弟高度的最大值，就可得到整棵树的高度。左孩子、右兄弟的高度，可通过递归求解。

因此，可按照后根遍历的顺序来计算树的高度。计算步骤如下：

（1） 计算左孩子子树的高度；

（2） 计算右兄弟子树的高度；

（3） 根据对树的高度的定义，取左孩子子树高度加 1 值与右兄弟子树高度的最大值，即为树在根结点处的高度。

具体代码如下。

算法 5.20 计算树的高度

```
int TreeDepth(Tree T)
{    // 计算树的高度
     if ( !T ) return 0;
     else
     {    h1 = TreeDepth( T->firstchild );
          h2 = TreeDepth( T->nextbrother);
          return (max(h1+1, h2));
     }
} // TreeDepth
```

5.5.4.3 输出森林中所有从根到叶子结点的路径

显然，可对森林进行先序遍历，并将访问到的结点存储在栈中，当访问到叶子结点时，输出栈中从根到叶子结点的路径。具体代码如下。

算法 5.21 输出森林中所有从根到叶子结点的路径

```
void AllPath(Tree &T, SeqStack &S)
{    // 输出森林中所有从跟到叶子结点的路径
     while ( !T )
     {    Push( S, T->data);
          if ( !T->firstchild ) PrintStack(S); // 到达一个叶子结点，输出路径
          else  AllPath( T->firstchild, S);
          Pop(S, T)                          // 回退到双亲结点
          T = T->nextbrother;                // 进入双亲的兄弟结点
     }
} // AllDepth
```

5.6 小结

本章讨论了在现实世界中广泛存在的一种典型的非线性数据结构——树，以及树的一种常用形式——二叉树。本章重点讨论了二叉树的抽象数据类型描述、存储结构与各种遍历算法的实现，最后介绍了一种二叉树的实际应用，即霍夫曼树的构造与霍夫曼编码。由于树可以转换成为一个唯一与之对应的二叉树，因此对树的操作都可以转化为对二叉树的操作。本章的重点是二叉树的 4 种遍历算法。遍历是二叉树的基本操作，也是最重要的操作，因为大多数对二叉树的操作都是基于一定顺序的遍历完成的。

通过本章的学习，读者要明确树这种数据结构与线性的数据结构之间的区别，熟练掌握树的几种遍历算法，并且要学会根据实际情况选择合适的遍历算法进行实现。

5.7 练习

1. 在具有 n(n>=1)个结点的 k 叉树中,用 k 叉链表表示，有（）个空指针

   A. k*n+1　　B. （k-1）*n+1

   C. k*n-1　　D. k*n

2. 在二叉树的二叉链表中，空指针个数等于非空指针个数加（）

A. 2　　B. 1
C. 0　　D. -1

3. 一棵深度为 k 且只有 k 个结点的二叉树按照完全二叉树的顺序存储方式存放于一个一维数组 R[n]中，那么 n 应至少是（ ）
A. 2k　　B. 2k+1
C. $2^k - 1$　　D. 2^{k-1}

4. 二叉树的叶结点在先序、中序、后序遍历过程中的相对顺序（）
A. 发生改变　　B. 不发生改变
C. 无法确定　　D. 以上均不正确

5. 二叉树的层次遍历，用（）结构较为合适
A. 栈　　B. 队列
C. 线性表　　D. 数组

6. 在一棵 n 个结点的线索二叉树中，线索的数目是（ ）(另外增加的头结点不计）。
A. n-1　　B. n+1
C. 2n　　D. 2n-1

7. 如果 T2 是由有序树 T 转换成的二叉树，那么 T 中结点的后根遍历顺序对应 T2 中结点的（）遍历顺序
A. 先序　　B. 中序
C. 后序　　D. 层次序列

8. 设 F 是一个森林，B 是由 F 转换得到的二叉树，F 中有 n 个非叶结点，则 B 中右指针域为空的结点有（）个。
A. n-1　　B. n
C. n+1　　D. n+2

9. 用 n 个权值构造出来的 Huffman 树有（）个结点.
A. 2n-1　　B. 2n
C. 2n+1　　D. n+1

10. 对于 n(n>=2)个权值不同的字符构造的 Huffman 树，下面描述错误的是（）
A. 该树一定是一棵完全二叉树　　B. 树中一定没有度为 1 的结点
C. 树中两个权值最小的结点一定是兄弟结点
D. 树中任何一个非叶结点的权值一定不小于与之对应的下一层任一结点的权值。

11. 1000 个结点的完全二叉树（编号从 1 开始），树的深度为____，第一个叶结点的编号为 ____，倒数第二层的叶结点个数为____。

12. 二叉树先序、中序和后续遍历的结果如下，请填上合适的字母。
先序：A ____ D E ____ F G
中序：D B ____ A F ____ C
后序：____ E B G F C ____

13. 编程实现二叉树的基本操作，用二叉链表结构实现。

14. 基于二叉树的遍历，实现以下功能：（1）二叉树深度；（2）叶子结点个数；（3）二叉树的繁茂度；（4）从根到所有叶子结点的路径；（5）交换二叉树中所有左右分支结点；（6）二叉树中两个互不为祖先的结点，寻找距离它们最近的共同祖先；（7）判断给定的二叉树是否是完全二叉树。

15. 已知一个二叉树的先序序列是 ABDCE，中序序列是 BDAEC，请问能否唯一地确定这个二叉树的结构？编程实现该二叉树的二叉链表示。

16. 设二叉树采用二叉链表存储，编写函数，对二叉树中每个元素值为 x 的结点，删除以它为根的子树，并释放相应空间。

17. 编写函数，对一棵以孩子-兄弟链表表示的树，输出第 i 层的所有元素。

18. 家谱管理系统

[问题描述]

实现具有下列功能的家谱管理系统。

[基本要求]

（1）输入文件以存放最初家谱中各成员的信息，成员的信息中均应包含以下内容：姓名、出生日期、婚否、地址、健在否、死亡日期（若其已死亡），也可附加其它信息、但不是必需的。

（2）实现数据的文件存储和读取。

（3）以图形方式显示家谱。

（4）显示第 n 代所有人的信息。

（5）按照姓名查询，输出成员信息（包括其本人、父亲、孩子的信息）。

（6）按照出生日期查询成员名单。

（7）输入两人姓名，确定其关系。

（8）某成员添加孩子。

（9）删除某成员（若其还有后代，则一并删除）。

（10）修改某成员信息。

（11）要求建立至少 40 个成员的数据，以较为直观的方式显示结果，并提供文稿形式以便检查。

（12）界面要求：有合理的提示，每个功能可以设立菜单，根据提示，可以完成相关的功能要求。

（13）存储结构：根据系统功能要求自行设计，但是要求相关数据要存储在数据文件中。测试数据：要求使用 1、全部合法数据；2、局部非法数据。进行程序测试，以保证程序的稳定。

19. 树的应用

[问题描述]

JSON (JavaScript Object Notation) 是一种轻量级的数据交换格式，可以用来描述半结构化的数据。JSON 格式中的基本单元是值 (value)，出于简化的目的本题只涉及 2 种类型的值：

* 字符串 (string)：字符串是由双引号 " 括起来的一组字符（可以为空）。如果字符串的内容中出现双引号 "，在双引号前面加反斜杠，也就是用 \" 表示；如果出现反斜杠 \，则用两个反斜杠 \\ 表示。反斜杠后面不能出现 " 和 \ 以外的字符。例如：""、"hello"、"\"\\"。

* 对象 (object)：对象是一组键值对的无序集合（可以为空）。键值对表示对象的属性，键是属性名，值是属性的内容。对象以左花括号 { 开始，右花括号 } 结束，键值对之间以逗号 , 分隔。一个键值对的键和值之间以冒号 : 分隔。键必须是字符串，同一个对象所有键值对的键必须两两都不相同；值可以是字符串，也可以是另一个对象。例如：{}、{"foo": "bar"}、{"Mon": "weekday", "Tue": "weekday", "Sun": "weekend"}。

除了字符串内部的位置，其他位置都可以插入一个或多个空格使得 JSON 的呈现更加美观，也可以在一些地方换行，不会影响所表示的数据内容。例如，上面举例的最后一个 JSON 数据也可以写成如下形式。

```
{
  "Mon": "weekday",
  "Tue": "weekday",
  "Sun": "weekend"
}
```

给出一个 JSON 格式描述的数据，以及若干查询，编程返回这些查询的结果。

[输入格式]

第一行是两个正整数 n 和 m，分别表示 JSON 数据的行数和查询的个数。

接下来 n 行，描述一个 JSON 数据，保证输入是一个合法的 JSON 对象。

接下来 m 行，每行描述一个查询。给出要查询的属性名，要求返回对应属性的内容。需要支持多层查询，各层的属性名之间用小数点 . 连接。保证查询的格式都是合法的。

[输出格式]

对于输入的每一个查询，按顺序输出查询结果，每个结果占一行。

如果查询结果是一个字符串，则输出 STRING <string>，其中 <string> 是字符串的值，中间用一个空格分隔。

如果查询结果是一个对象，则输出 OBJECT，不需要输出对象的内容。

如果查询结果不存在，则输出 NOTEXIST。

[样例输入]

```
10 5
{
"firstName": "John",
"lastName": "Smith",
"address": {
"streetAddress": "2ndStreet",
"city": "NewYork",
"state": "NY"
},
"esc\\aped": "\"hello\""
}
firstName
address
address.city
address.postal
esc\aped
```

[样例输出]
STRING John
OBJECT
STRING NewYork
NOTEXIST
STRING "hello"
[基本要求]
（1）要求从文本文件中输入；
（2）本题是一棵普通的树（即每个结点的孩子数不固定，不能单纯采用 n 叉树来解决），可以考虑使用孩子兄弟表示法等进行表示和存储；
（3）严格按照要求的输入输出格式进行数据的输入、输出；
（4）选做：使用图形界面（或字符格式化摆成的树形结构，参考 Linux 下的 tree 命令），以树状形式显示输入的 JSON 格式数据。

5.8 延伸阅读

1952 年在美国计算机科学家 Grace M. Hopper 开发的编译器系统 A-1 中，使用了二叉树来表达、存储以及处理代数公式[1]。

1952 年 David A. Huffman 在麻省理工学院攻读博士学位，其导师 Robert Fano 给出的研究课题是查找最有效的二进制编码。Huffman 使用自底向上的方法构建二叉树，并证明了其最优性[2]，而之前的 Shannon-Fano coding 编码则采用自顶向下的方式构建树[3]，无法生成最优编码且时间效率低。

1968 年 Donald E. Knuth 在其著作 *The Art of Computer Programming* 中介绍了二叉树的三种遍历方案：preorder、postorder 和 end-order，而在 1973 年该书第二版中则把 end-order 改为 inorder（symmetric order）。Knuth 在书中描述了遍历方案的递归与非递归算法，同时抛出问题：是否可以设计出不使用栈（stack-free）、不使用标签（tag-free）且非递归（non-recursive）的遍历算法。1979 年 James H. Morris 提出了一种新的非递归算法[4]，该算法不需要栈且不需要在结点中添加标签就能实现对二叉树的遍历。

1959 年 Rene D. L. Briandais 提出了 Trie 树的概念[5]。1960 年 Edward Fredkin 完成了前缀树的最初设计[6]，并从英文单词 retrieval 中提取出 trie 作为其名称。1973 年 Peter Weiner 最先提出构造后缀树及后缀自动机的线性时间算法[7]，后来 Esko Ukkonen 设计出更简单且运行效率更高的算法[8]。

1975 年悉尼大学的 Ross Quinlan 最早提出构建决策树的 ID3 算法[9]，并在 1993 年提出了 ID3 算法的改进算法 C4.5[10]。1984 年美国国家科学院院士 Leo Breiman 等人设计出 CART 算法[11]，其中文名称为分类回归树算法。

参考文献：
[1] HOPPER G M, The education of a computer[C]. Proceedings of the 1952 ACM national meeting (Pittsburgh). 1952: 243-249.
[2] HUFFMAN D A. A method for the construction of minimum-redundancy codes[C]. Proceedings of the IRE, 1952, 40(9): 1098-1101.
[3] FANO R M. The transmission of information[M]. Cambridge, MA, USA: Massachusetts Institute of Technology, Research Laboratory of Electronics, 1949.

[4] MORRIS J M. Traversing binary trees simply and cheaply[J]. Information Processing Letters. 1979，9 (5): 197-200.
[5] BRIANDAIS R D L, File searching using variable length keys[C]. Proceedings of the AFIPS Western Joint Computer Conference. San Francisco, 1959: 295-298.
[6] FREDKIN E. Trie memory[J]. Communications of the ACM, 1960, 3 (9): 490-499.
[7] WEINER P, Linear pattern matching algorithms[C]. IEEE Conference Record of 14th Annual Symposium on Switching and Automata Theory, Iowa City, 1973: 1-11.
[8] UKKONEN E. On-line construction of suffix trees[J]. Algorithmica, 1995, 14(3): 249-260
[9] QUINLAN R. Induction of decision trees[J]. Machine Learning, 1986, 1(1): 81-106
[10] QUINLAN R. C4.5: programs for machine learning[M]. San Francisco: Morgan Kaufmann Publisher, 1993.
[11] BREIMAN L, FRIEDMAN J H, OLSHEN R A, et al. Classification and regression trees[M]. Monterey: Brooks / Cole Publishing, 1984.